



**Centro académico de Limón**

**Escuela de Computación Lenguajes de programación**

**Proyecto Programado #3**

**Videojuego: Atlas**

**Código de curso: IC4700**

**Estudiante:**

**Alice Arias Salazar | 2023104639**

**Heldyis Agüero Espinoza | 2023296812**

**Yeremi Calvo Porras | 2023083332**

**Profesor:**

**Allan Rodríguez Davila**

**Grupo: 60**

**I Semestre 2026**

## Índice

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Manual de usuario y pruebas de funcionalidad.....</b>   | <b>3</b>  |
| 1.1      | Abrir el proyecto en VSCode y ejecutar la aplicación ..... | 3         |
| 1.2      | Menú principal.....                                        | 4         |
| 1.3      | Pantalla de juego.....                                     | 5         |
| 1.4      | Mapa y módulos.....                                        | 5         |
| 1.5      | Bitácora en tiempo real .....                              | 6         |
| 1.6      | Ficha de módulo.....                                       | 7         |
| 1.7      | Controles de misión .....                                  | 7         |
| 1.8      | Movimiento rápido .....                                    | 8         |
| 1.9      | Usuario e inventario .....                                 | 9         |
| 1.10     | Tripulantes y objetivo actual .....                        | 9         |
| 1.11     | Mensaje de victoria.....                                   | 10        |
| <b>2</b> | <b>Descripción del problema .....</b>                      | <b>10</b> |
| <b>3</b> | <b>Diseño del programa .....</b>                           | <b>11</b> |
| 3.1      | Backend .....                                              | 11        |
| <b>4</b> | <b>Análisis de Resultados .....</b>                        | <b>12</b> |
| <b>5</b> | <b>Análisis de predicados .....</b>                        | <b>13</b> |
| 5.1      | Forzar gane .....                                          | 13        |
| 5.2      | Restaurar estado de partida .....                          | 13        |
| 5.3      | Cómo gano.....                                             | 14        |

## 1 Manual de usuario y pruebas de funcionalidad

Primero deben clonar el repositorio desde GitHub usando GitHub Desktop: <https://github.com/Alice-Arias/PY3-Operaci-n-Atlas.git>

Esto descargará todos los archivos en una carpeta local en su computadora.

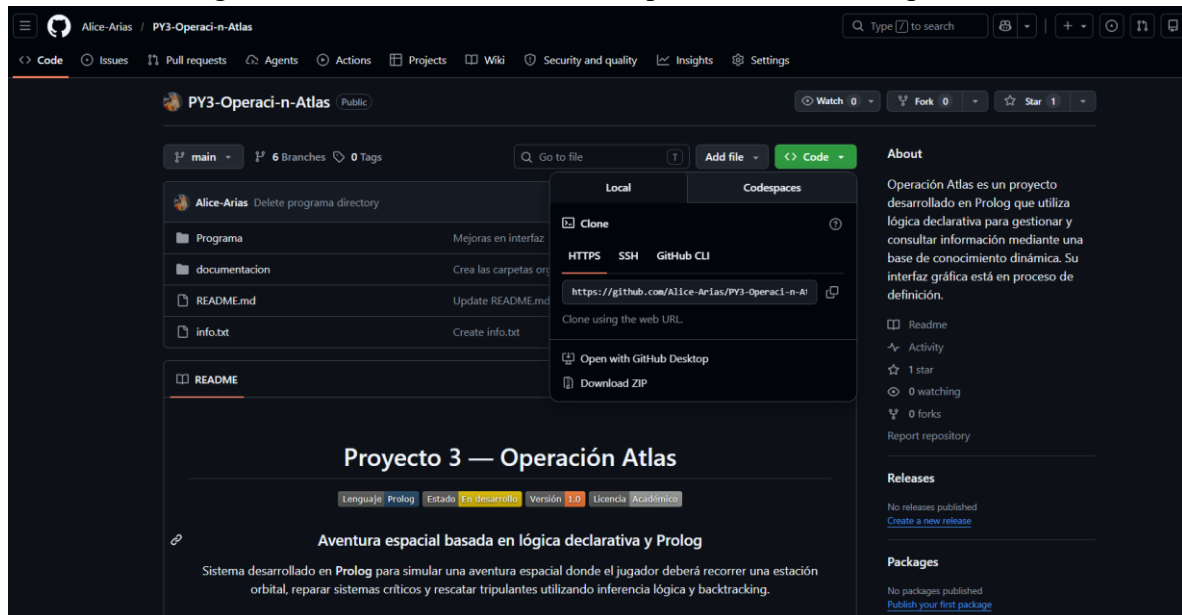


Figura 1

Si prefiere hacerlo de otra manera, puedes ingresar al repositorio y hacer clic en Code. Luego, como se muestra en la Figura 1, selecciona la opción Download ZIP. Una vez que la descarga se complete, descomprime el archivo y estarás listo para continuar con el siguiente paso.

### 1.1 Abrir el proyecto en VSCode y ejecutar la aplicación

Una vez descargado, abran Visual Studio Code y carguen la carpeta del proyecto vamos a abrir una terminal nueva, esto para comenzar a compilar el proyecto. Para ello, debe colocarse en el fichero “*controller*” y ejecutar el comando “*node server.js*”. En otra terminal paralela debe colocarse en el fichero “*frontend*” y ejecutar el comando “*npm run dev*”

```
hellyd@HellydisAE MINGW64 ~/Desktop/PY3_Lenguajes/programa (main)
$ cd frontend
hellyd@HellydisAE MINGW64 ~/Desktop/PY3_Lenguajes/programa/frontend (main)
$ npm run dev
> programa@0.0.0 dev
> vite
7:13:00 p. m. [vite] (client) Re-optimizing dependencies because lockfile has changed
VITE v8.0.14 ready in 429 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Figura 3

```
hellyd@HellydisAE MINGW64 ~/Desktop/PY3_Lenguajes/programa (main)
$ cd controller
hellyd@HellydisAE MINGW64 ~/Desktop/PY3_Lenguajes/programa/controller (main)
$ node server.js
=====
SERVIDOR NODE + PROLOG ACTIVO
PUERTO: 3000
=====
```

Figura 2

Esta combinación de comandos hace posible ejecutar la aplicación de forma local en su dispositivo, se vincula el backend, que corresponde a la lógica de negocio del sistema, con el frontend donde se encuentran los componentes visuales que hacen posible la interacción directa y sencilla entre el usuario y el sistema.

El ejecutar estos dos comandos, se debe presionar el link brindado tras la ejecución de “npm run dev”, esto abrirá una ventana en su navegador asignado que es donde se estará ejecutando y mostrando la aplicación.

## 1.2 Menú principal



Figura 4

Esta es la primera página que verá el usuario luego de ejecutar el sistema, lo que se muestra es el título del videojuego acompañado de una tarjeta de opciones. Esta tarjeta de opciones cuenta con 3 componentes visuales: Un input, un botón para iniciar la partida y otro botón para cargar partida.

El usuario notará que los botones al inicio están desactivados, este es un método de seguridad por parte del sistema para asegurar que el usuario ingrese un nombre antes. El nombre del usuario es de suma importancia ya que permite guardar las partidas posteriormente.

El botón de inicio de partida se activa luego de que el usuario haya escrito al menos tres caracteres en el input, esto se debe simplemente a una estandarización para el manejo de archivos correspondientes a las partidas. El botón de para cargar partida se activa únicamente en caso de que el nombre de usuario escrito en el input corresponda a uno que ya exista en el sistema, es decir, hay partidas guardadas de ese usuario previamente.

Como complemento informativo, al momento de escribir se despliega un pequeño span bajo el input que le permite al usuario saber si le faltan caracteres a su nombre, si se está buscando una partida guardada y si tiene o no tiene partidas guardadas disponibles.

### 1.3 Pantalla de juego

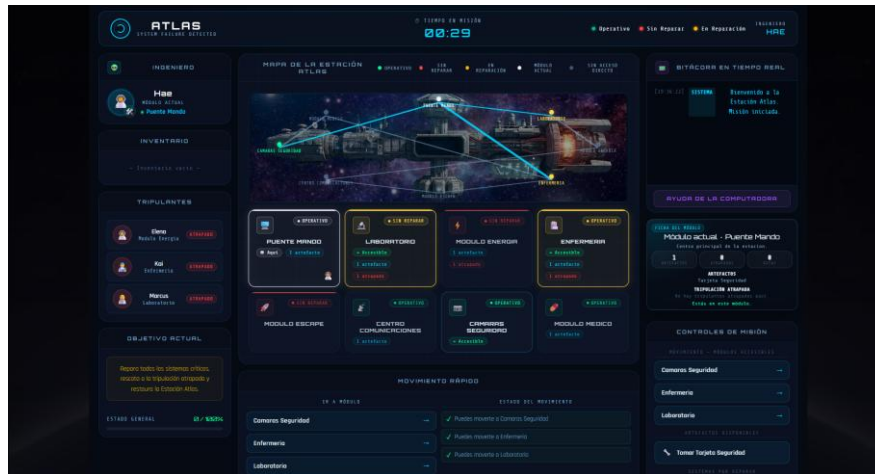


Figura 5

Esta es la página que se le muestra al usuario cuando se ingresa a una partida. En esta interfaz compuesta el usuario puede disponer de diferentes tipos de información, asimismo como controles y botones interactivos destinados a facilitar el uso del juego para el usuario.

A continuación, debido a la cantidad de secciones disponibles en esta página, se explicará cada sección de forma separada.

### 1.4 Mapa y módulos



Figura 6

Esta es la sección central en la página de juego. En este apartado, como elemento más destacado, se encuentra renderizado una simulación del mapa en tiempo real de los módulos de la nave espacial del videojuego. Cada módulo es representando en el mapa como un punto, la conexión y comunicación entre el módulo es representada con líneas.

El color de las líneas representa un estado diferente, el usuario puede conocer el significado de cada color en la parte superior de este componente donde se indica que el color verde representa el estado operativo, el rojo sin reparar, el amarillo está en reparación, el punto blanco indica la posición actual del usuario y las líneas y puntos si color indican secciones sin acceso directo.

En la parte inferior de este componente se presentan los módulos de la nave agrupados en tarjetas con información pertinente de cada uno de ellos: Su estado, nombre y objetivos disponibles.

## 1.5 Bitácora en tiempo real

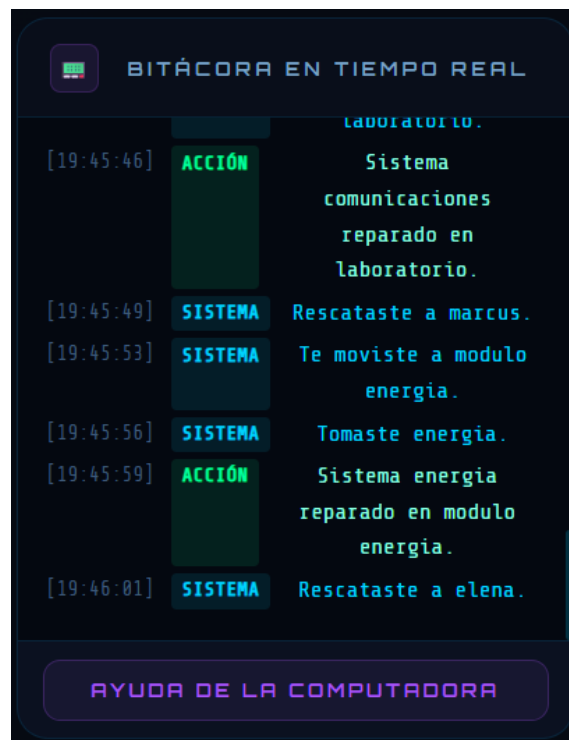


Figura 7

Este componente sirve como recopilador de sucesos del videojuego, su objetivo es brindarle al usuario la posibilidad de estar al tanto en todo momento de cada acción previamente realizada o de saber por qué falló el intento de una acción.

En este componente también se encuentra el botón de ayuda de la computadora, el objetivo de este botón es sugerir movimiento y acciones al usuario en caso de que este necesite la ayuda respectiva. Las sugerencias, consejos e información brindada se desplegarán en la bitácora.

## 1.6 Ficha de módulo



Figura 8

En esta sección se representa la información específica del módulo donde se ubica específicamente en ese momento el usuario. El componente dispone de diferentes tipos de información relacionada con el estado del módulo de la nave.

En esta sección de la interfaz el usuario puede visualizar los artefactos disponibles para recolección en ese módulo, los tripulantes atrapados y las posibles rutas que el usuario puede tomar a partir de esa ubicación específica.

## 1.7 Controles de misión



Figura 9

En esta sección se le ofrece al usuario un panel de control que le sirve para desplazarse

entre módulos y llevar a cabo diferentes acciones dentro del videojuego.

En la parte superior del componente el usuario dispone de botones dinámicos que representan los módulos a los que puede desplazarse.

En el bloque medio el usuario dispondrá, momentáneamente, de botones que le permitirán conseguir los artefactos disponibles en ese módulo y reparar sistemas.

La parte inferior del componente renderiza botones en función de los tripulantes a los cuáles se puede acceder para ser rescatados.

Sin embargo, es importante apuntar que estos botones no representan posibles 100% reales, es decir que existe la posibilidad de que aparezca un botón para rescatar a un tripulante, sin embargo, si no se tienen los artefactos o secciones reparadas necesarias esta acción no podrá ser llevada a cabo. En caso de que se de alguno de estos conflictos, el usuario puede percatarse de ellos en la sección de bitácora en tiempo real.

En la parte más baja de esta sección se dispone de dos botones que ofrecen la opción de guardar la partida y salir de la misma.

## 1.8 Movimiento rápido



Figura 10

La sección de movimiento rápido representa una sección agilizada y optimizada de la sección de controles de misión para el movimiento del usuario entre módulos. En esta sección el usuario puede previsualizar las posibilidades de desplazamiento que tiene, esto le ahorra al usuario, en caso de que desee jugar de esta manera, equivocarse constantemente en su movilización por la nave espacial del videojuego.

Además, en esta sección también se encuentra el botón de forzar gane. Este botón ofrece al usuario la posibilidad de completar la partida de forma automática, en caso de que haya conflicto con algún módulo por incompletitud de requisitos el sistema lo informa.



## 1.9 Usuario e inventario



Figura 11

Corresponde a una sección de información simple en la que se muestra el nombre ingresado por el usuario antes de ingresar a la partida, el módulo donde se encuentra y los objetos que ha obtenido hasta ese momento.

## 1.10 Tripulantes y objetivo actual



Figura 12

Muestra una representación de los tripulantes en la nave y su estado actual, de esta

manera el usuario puede monitorear el progreso de su partida en cuanto al rescate de los tripulantes se refiere.

En la sección de objetivo actual se muestra una leyenda que describe al usuario de manera genera las tareas que tiene pendientes para ganar la partida, en la parte inferior se muestra una barra de progreso que aumenta conforme se avanza en la partida y se cumplen progresivamente las condiciones de victoria.

### 1.11 Mensaje de victoria



Figura 13

Cuando el usuario completa todas las tareas pendientes en la nave gana la partida, en ese momento se le despliega la renderización de una alerta que le indica que ha ganado la partida. Asimismo, se le muestra el tiempo de la partida y tres botones con diferentes funcionalidades.

Al usuario se le ofrece la posibilidad de guardar la misión para reproducir su repetición después, el botón de ver la bitácora de tiempos le muestra el estado final de la bitácora mostrándole todo el progreso cronológico de la misión y finalmente un botón inferior para que el usuario pueda iniciar otra partida de forma más rápida.

## 2 Descripción del problema

El problema consiste en modelar y resolver una aventura espacial interactiva mediante el paradigma lógico en Prolog, situando al jugador en el rol de Ingeniero Principal de Abordo dentro de la Estación Aurora, una base orbital científica cuyo sistema central sufrió daños críticos y cuyos módulos quedaron aislados tras una severa tormenta solar.

La problemática central exige que el usuario navegue de forma lógica a través de secciones interconectadas de la nave para reparar sistemas vitales en estado de fallo, recolectar y utilizar artefactos específicos, y rescatar a la tripulación atrapada. La complejidad del reto radica en que todas las acciones del personaje están sujetas a un estricto conjunto de restricciones dinámicas del dominio, tales como la existencia de enlaces bidireccionales entre áreas, la necesidad de poseer ciertos objetos para desbloquear accesos de seguridad, el cumplimiento de pasos previos de exploración y la dependencia de que ciertos servicios estén totalmente restaurados antes de permitir un rescate o una evacuación. De este modo, el sistema prescinde de un enfoque imperativo tradicional y se define completamente a través de inferencias, hechos y reglas lógicas que validan el estado inicial, controlan el flujo mediante backtracking y determinan las condiciones simultáneas de

victoria, conectando toda esta base de conocimiento a una interfaz gráfica que sirve como canal de comunicación con el usuario.

### 3 Diseño del programa

Para el desarrollo de este proyecto, se utilizó lenguaje lógico Prolog apoyado en la librería React JS para la implementación de la interfaz gráfica de usuario requerida para esta aplicación.

Como se utilizó React, se optó por la utilización de los servicios de Node Js como interfaz entre el backend (Las funciones en Prolog) y el frontend (El proyecto de React).

Para la estructura organizacional del proyecto se llevó a cabo una estructura básica de modelo – vista – controlador, siendo vista la carpeta frontend, modelo la carpeta backend y controlador la carpeta controller, es en esta última donde se encuentra la lógica de nodejs para comunicar el backend con el frontend

#### 3.1 Backend

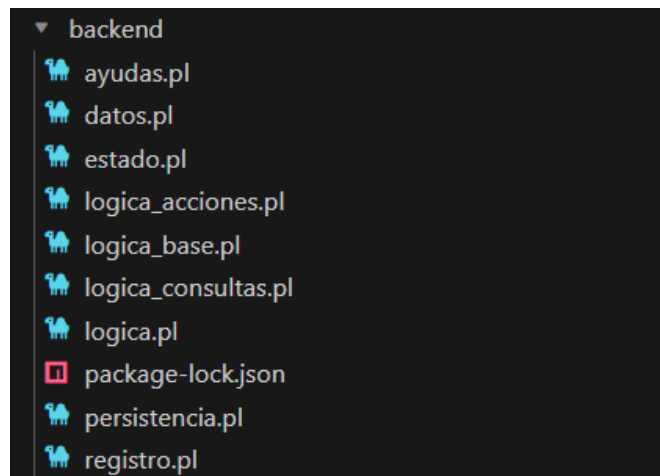


Figura 14

Estos son los módulos de backend desarrollados, cada módulo representa una sección de la lógica de negocio para el funcionamiento óptimo de la aplicación.

En el primer módulo llamado **ayudas.pl** se desarrollaron los predicados de apoyo necesarios en la lógica principal del proyecto. Aquí se encuentran, por ejemplo, predicados dedicados a la validación de datos de entrada, impresión de mensajes de error o advertencia y transformaciones de datos.

**datos.pl** se encarga de almacenar todos los datos estáticos del sistema, es en este módulo donde se definieron todos los hechos con los que se jugarán. Además, también contiene predicados dedicados a la conservación de datos en el tiempo.

El siguiente módulo se denomina **estado.pl**, en este módulo se concentran las funciones de actualización de estado durante la partida, es decir, se encarga de obtener el estado nuevo y reemplazarlo por el antiguo. Este módulo es de suma importancia para el sistema ya que permite actualizar la información en tiempo real y además dar seguimiento a los movimientos de los jugadores.

En el módulo **logica\_acciones.pl** se encuentran todos los predicados destinados a manipular y gestionar la lógica de cada acción disponible en el juego, es aquí donde, por

ejemplo, se implementa la lógica que permite rescatar a un tripulante o reparar un sistema.

Por otro lado, en el módulo *logica\_base.pl* se encuentra la lógica de validación de movimientos y acciones, es decir, este módulo se encarga directamente de manipular la lógica de la dificultad de este videojuego, que es básicamente seguir la secuencia adecuada de acciones que le permitan al jugador cumplir con los objetivos establecidos.

*logica\_consultas.pl* contiene también lógica de verificación, pero en este caso, se encarga de verificar las condiciones de victoria, es decir, este módulo es el encargado en todo momento de verificar que el usuario haya rescatado a todos los tripulantes, que todos los sistemas estén reparados y todas las demás condiciones de victoria establecidas en el sistema.

En el caso de módulo *logica.pl* funciona simplemente como una interfaz de conexión entre los otros módulos de lógica.

El siguiente es el módulo de *persistencia.pl* este módulo implementa el sistema de guardar y cargar partidas. Su objetivo es tomar todo el estado actual del juego, almacenarlo en un archivo y luego poder recuperarlo exactamente como estaba.

Por último, el módulo de *registro.pl* administra el registro y seguimiento de las partidas de los jugadores. Se encarga de generar identificadores únicos para nuevas partidas, asociar cada partida con un archivo de guardado, mantener un registro persistente de las partidas existentes y permitir la recuperación de partidas previamente guardadas. Además, ofrece funcionalidades para consultar partidas pendientes de finalizar, iniciar nuevas partidas y cargar partidas mediante su identificador. El registro de partidas se almacena de forma persistente en disco dentro de la carpeta destinada a los archivos de guardado.

#### 4 Análisis de Resultados

| Objetivo                                             | Alcanzado | No alcanzado |
|------------------------------------------------------|-----------|--------------|
| Modelo de base de datos/reglas.                      | X         |              |
| Implementación de lógica de movimiento y navegación  | X         |              |
| Gestión de artefactos: tomar y usar                  | X         |              |
| Restricciones de acceso, estados y pasos             | X         |              |
| Implementación de reparaciones de sistemas           | X         |              |
| Implementación de lógica para rescate de tripulantes | X         |              |
| Validación de condiciones de victoria                | X         |              |
| Implementación de interfaz grafica                   | X         |              |
| Funcionalidad extra: guardar y ver repetición        | X         |              |

## 5 Análisis de predicados

### 5.1 Forzar gane

El predicado *forzar\_gane\_plan* fue implementado como un mecanismo avanzado de asistencia que se encarga de analizar el estado actual de la partida y generar un plan estructurado para alcanzar la victoria, en caso de que el usuario no pueda encontrar una solución por su cuenta. Su funcionamiento se basa en la evaluación de las condiciones de éxito del juego, verificando cuáles sistemas aún requieren reparación, qué tripulantes permanecen sin rescatar y cuáles artefactos todavía no han sido recolectados. A partir de esta información, el predicado identifica los objetivos pendientes y construye una serie de acciones necesarias para completar la misión.

El sistema se apoya en la información de sistemas, tripulantes, artefactos y restricciones de acceso entre módulos para elaborar recomendaciones concretas. Posteriormente, estas recomendaciones son procesadas mediante algoritmos de búsqueda de rutas que permiten expandir cada objetivo en una secuencia detallada de desplazamientos y acciones. Como resultado, el jugador recibe un plan ordenado que no solo indica qué debe hacer, sino también cómo llegar a cada destino para lograrlo.

El enfoque utilizado permitió desarrollar una funcionalidad adaptable a cualquier estado de la partida, ya que el plan se genera dinámicamente a partir de la información disponible en el momento de la consulta. Esto mejora la utilidad de la asistencia proporcionada al jugador y facilita la toma de decisiones para completar la misión de manera eficiente.

### 5.2 Restaurar estado de partida

El predicado *restaurar\_estado\_partida* fue implementado como el componente principal del sistema de persistencia del juego. Su responsabilidad consiste en reconstruir completamente una partida previamente guardada a partir de una estructura que contiene toda la información necesaria para continuar la sesión. Para lograrlo, el predicado recibe un término con el estado almacenado de la partida, incluyendo la posición del jugador, el inventario, los módulos visitados, los artefactos obtenidos, los sistemas registrados y el estado de la tripulación.

El proceso de restauración inicia eliminando cualquier información existente en la base de conocimiento dinámica mediante el uso de *retractall*, garantizando que no permanezcan datos residuales de una partida anterior. Posteriormente, el sistema reconstruye cada uno de los hechos dinámicos utilizando *assertz*, restaurando tanto la información general de la partida como las colecciones de artefactos, sistemas y tripulantes. Para las estructuras que contienen múltiples elementos se emplea *forall*, permitiendo recorrer las listas almacenadas e insertar nuevamente cada hecho en la base de conocimiento.

Este enfoque permite recuperar de forma íntegra el estado del juego en el momento exacto en que fue guardado, asegurando la consistencia de los datos y permitiendo que el jugador continúe la partida sin pérdida de progreso. Además, al centralizar toda la lógica de restauración en un único predicado, se facilita el mantenimiento del sistema y la incorporación de nuevos elementos al estado persistente en futuras versiones del proyecto.

### 5.3 Cómo gano

El predicado *como\_gano* fue implementado como un sistema de asistencia estratégica cuyo objetivo es orientar al jugador sobre las acciones necesarias para completar la misión. Su funcionamiento consiste en analizar el estado actual de la partida y determinar cuáles elementos aún se encuentran pendientes para alcanzar la victoria. Para ello, recopila información sobre los sistemas que no han sido restaurados, los tripulantes que permanecen sin rescatar y los artefactos que todavía no han sido obtenidos. Además, evalúa la posición actual del jugador y las conexiones entre módulos para identificar tanto los movimientos disponibles como aquellos que se encuentran bloqueados por requisitos aún no cumplidos.

A partir de este análisis, el predicado genera una serie de recomendaciones concretas destinadas a facilitar el progreso dentro del juego. Estas recomendaciones incluyen la obtención de artefactos necesarios, la reparación de sistemas, el rescate de tripulantes y la visita a módulos que permiten desbloquear nuevas áreas. Posteriormente, todas las sugerencias son unificadas y depuradas para eliminar repeticiones, produciendo una lista organizada de pasos que se presenta al jugador de forma clara y numerada. Gracias a este enfoque, la ayuda ofrecida se adapta dinámicamente al estado de la partida, proporcionando una guía personalizada que facilita la toma de decisiones y el cumplimiento de los objetivos restantes.

A simple vista puede parecer que este predicado cumple la misma función que *forzar\_gane*, sin embargo, sus objetivos son completamente distintos. Como se destacó antes, *como\_gano* ofrece una funcionalidad pasiva dentro del sistema, asesorando al usuario sobre los pasos que necesita ir siguiendo para alcanzar la victoria, a diferente de *forzar\_gane*, este predicado no ejecuta acciones por su cuenta, solo entrega recomendaciones.